

The AGE Predictor

How it works, and how it fits our grokking experiment

Thesis: *A Unified Benchmark of Grokking Predictors in Neural Networks*

Predictor 4 of 9 — Status: closed (negative result)

The simple meaning. AGE stands for *Adaptive Grokking Epoch via Neural Collapse*. It measures one thing. Do the internal features of examples with the same answer come close together? If yes, the model has organised its features neatly. AGE then asks whether this neatness appears *before* the model groks. In our experiment, it does not. It appears *after*.

Contents

1	Why we look at this predictor	1
2	The experimental setting (the substrate)	1
3	What AGE measures: the NC1 ratio	2
3.1	The raw material: feature vectors	2
3.2	Two kinds of spread	2
3.3	The ratio	2
3.4	A small worked example	3
3.5	A picture from everyday life	3
4	How AGE is computed in the code	3
4.1	Collecting the features	3
4.2	Computing the statistics	4
4.3	Apple Silicon (MPS) detail	4
4.4	Properties of the signal	4
5	How AGE fits into the experiment pipeline	4
6	The test: does AGE lead the grok epoch?	5
7	Results on the 5 seeds	5
8	Interpretation, and an honest caveat	6
8.1	What the result means	6
8.2	The caveat about the argmin	6
8.3	A reading we have not tried	7

9 Where AGE stands in the thesis

7

10 Summary

7

1 Why we look at this predictor

Let us first see why this idea came into the benchmark at all.

Papayan, Han and Donoho (2020) studied networks trained until the training error is zero and beyond. They observed a pattern, which they called **Neural Collapse**. It has several parts. The first part is called **NC1**, the *within-class variability collapse*. In simple terms, the last-layer features of examples from the same class become almost identical. All of them sit at one point, the class mean.

Now, grokking is a case where the model first memorises the training set and then, much later, generalises. So a natural question comes up. Does the feature space become tidy (collapse) at the moment of generalisation, or before it? If the collapse happens *early*, we can watch it and predict the grokking epoch. This is the hypothesis behind AGE.

The idea is supported by related work. Beaglehole (2024) and Mallinar (2024) connect feature learning with the Average Gradient Outer Product (AGOP). Paul and Rupa (2026) read the collapse epoch as a marker of the transition. We are not repeating their theory here. We are only testing the practical claim on our own setup: does the minimum of NC1 lead the grok epoch?

2 The experimental setting (the substrate)

AGE is not run on its own. It sits on top of the common benchmark experiment, called **Nanda-Unified**. Every predictor in the thesis uses this same substrate. So let us fix the details once.

Item	Value
Task	$(a + b) \bmod 113$, vocabulary size 114, “=” token is 113
Model	1-layer transformer, 4 heads, $d_{\text{model}} = 128$, $d_{\text{mlp}} = 512$, no LayerNorm
Initialisation	small, $\mathcal{N}(0, 0.8/\sqrt{128} \approx 0.0707)$
Optimiser	AdamW, learning rate 10^{-3} , weight decay 1.0, $\beta = (0.9, 0.98)$
Training	full batch, 40,000 epochs
Data split	30% train, 70% test
Seeds	5

The training set has $N = \lfloor 0.3 \times 113^2 \rfloor = 3830$ examples. The **grok epoch** is the first epoch at which test accuracy is above 0.9. For the five seeds, the grok epochs are:

Seed	0	1	2	3	4
Grok epoch	14474	6988	10418	10193	24021

The mean is 13218.8 and the standard deviation is 5900.6. The spread is large. This is important, because a good predictor must follow this spread seed by seed.

3 What AGE measures: the NC1 ratio

3.1 The raw material: feature vectors

For every training example i , we take the input to the model's `output_head`, at sequence position 2. Position 2 is the “=” token, where the model produces its answer. This gives a feature vector $h_i \in \mathbb{R}^{128}$. Stacking all examples, we get a matrix

$$\Phi \in \mathbb{R}^{N \times 128}, \quad N = 3830.$$

Each example also has a label $y_i \in \{0, 1, \dots, 112\}$, which is the correct answer $(a + b) \bmod 113$. Each label is one *class*. So we have $C = 113$ classes.

3.2 Two kinds of spread

We need two averages.

- μ_c is the mean feature vector of class c .
- μ_G is the mean feature vector of all examples together (the global mean).

Within-class scatter tells us how spread out each class is around its own mean:

$$\text{Tr}_W = \frac{1}{C} \sum_c \frac{1}{n_c} \sum_{i \in c} \|h_i - \mu_c\|^2.$$

Between-class scatter tells us how far the class means are from the global mean:

$$\text{Tr}_B = \frac{1}{C} \sum_c \|\mu_c - \mu_G\|^2.$$

Here n_c is the number of examples in class c . The classes that have no training example are skipped.

3.3 The ratio

AGE uses the ratio of the two:

$$\text{NC1} = \frac{\text{Tr}_W}{\text{Tr}_B + 10^{-12}}$$

The tiny 10^{-12} only avoids division by zero. It has no effect otherwise.

How to read NC1. Large Tr_W means each class is spread out. Small Tr_B means the classes are close to each other. Either of these makes NC1 large. NC1 becomes small only when the classes are tight *and* far apart. Perfect collapse means $\text{NC1} \rightarrow 0$.

3.4 A small worked example

Let us take two classes with three points each, and only two numbers per point (in our model it is 128).

Class A: (1, 2), (3, 2), (2, 5) Class B: (8, 8), (10, 8), (9, 11)

Step 1: class means.

$$\mu_A = (2, 3), \quad \mu_B = (9, 9).$$

Step 2: within-class scatter. For class A, the differences from the mean are $(-1, -1)$, $(1, -1)$ and $(0, 2)$. Their squared lengths are 2, 2 and 4. The average is $8/3$. Class B has the same shape, so it also gives $8/3$. Hence

$$\text{Tr}_W = \frac{8}{3} \approx 2.67.$$

Step 3: global mean and between-class scatter. All six points together give $\mu_G = (5.5, 6)$. Then

$$\|\mu_A - \mu_G\|^2 = 3.5^2 + 3^2 = 21.25, \quad \|\mu_B - \mu_G\|^2 = 21.25,$$

so $\text{Tr}_B = 21.25$.

Step 4: the ratio.

$$\text{NC1} = \frac{2.67}{21.25} \approx 0.125.$$

If all three points of each class moved to their class mean, Tr_W would become 0, and so would NC1. This is exactly what “collapse” means.

3.5 A picture from everyday life

Think of two dartboards. Each player throws three darts at his own board. Tr_W is how scattered the darts are around each player’s own cluster. Tr_B is how far apart the two clusters are. A good result is tight clusters that are far from each other. Then NC1 is small.

4 How AGE is computed in the code

The implementation is in `src/predictors/age.py`. Let us go through it in the order the code runs.

4.1 Collecting the features

The function `_collect_representations_and_labels` registers a forward hook on `model.output_head`. A forward hook is a small function that PyTorch calls every time that layer runs. The hook captures the layer’s input at sequence position 2 (`inputs[0][:, 2, :]`). It returns two things:

- `Phi`, of shape $[N, 128]$, in float64 on the CPU;
- `Y`, of shape $[N]$, the labels.

4.2 Computing the statistics

The function `compute_age_metrics_for_checkpoint` loops over all classes. The number of classes is read from the model as $p = \text{output_head.out_features} - 1 = 113$. The core of the calculation is this:

```

mu_G = Phi.mean(dim=0)
for c in range(p):
    mask = (Y == c); n_c = int(mask.sum())
    if n_c == 0: continue
    Phi_c = Phi[mask]; mu_c = Phi_c.mean(dim=0)
    within_sq.append(((Phi_c - mu_c) ** 2).sum(dim=1).mean())
    between_sq.append(((mu_c - mu_G) ** 2).sum())
tr_w = float(torch.stack(within_sq).mean())
tr_b = float(torch.stack(between_sq).mean())
nc1 = tr_w / (tr_b + _EPS)
fn = float(Phi.norm(dim=1).mean())

```

Please match this with the formulas of Section 3. `within_sq` holds the inner average for each class, and `tr_w` is the outer average over classes. `between_sq` holds $\|\mu_c - \mu_G\|^2$, and `tr_b` is its average.

The code also records the **feature norm**

$$fn = \frac{1}{N} \sum_i \|h_i\|_2.$$

This is not part of the predictor. We keep it only as a diagnostic, and it will be useful in Section 7.

4.3 Apple Silicon (MPS) detail

The benchmark targets PyTorch on Apple Silicon. MPS does not support float64. So the features are first moved to the CPU, and only then converted with `.double()`. The order is important. If we call `.double()` first, it fails on MPS.

4.4 Properties of the signal

- It needs **no weight SVD**. It uses only features.
- It is **checkpoint-only**. It reads saved models and does not touch the training loop.
- It is **loss-agnostic**. It does not depend on the training loss or the optimiser.

5 How AGE fits into the experiment pipeline

AGE is a *checkpoint-only plugin* in `scripts/run_nanda_benchmark.py`. It is registered in two places:

```
CHECKPOINT_PREDICTOR_FUNCS["age"] and PREDICTOR_SUMMARY_KEY["age"]="age_predictor".
```

The function that runs it is `_checkpoint_predictor_age`. For each seed, the flow is as follows.

1. Load the 24 saved checkpoints `model_epoch_{e}.pt`. The epochs are log-spaced: 0, 1, 2, 3, 5, 9, 15, 24, 39, 62, 99, 158, 251, 398, 632, 1002, 1589, 2520, 3995, 6333, 10040, 15917, 25232, 39999.
2. Read the seed from `seed.npy` and re-seed the random generators.
3. Rebuild the training data with `get_dataloaders(number=p, batch_size=int(0.3*p*p))`. This makes the training split identical to the one used during training.
4. For each checkpoint, compute NC1 and the feature norm.
5. Save the results in `seed_N/age/`: `age_checkpoints.npy`, `age_nc1.npy`, `age_fn.npy` and `age_signal.json`.

6. Add an `age_predictor` block to `summary.json`.

The saving function `save_age_data` lives in `unified_measurements.py`. The live training loop is untouched. So AGE cannot disturb the training in any way.

Shared model versus predictor. `transformer.py` is the shared model file used by training, all predictors and evaluation. AGE reads this model, but it does not own it. The AGE code is only in `age.py` and its plugin function.

6 The test: does AGE lead the grok epoch?

From the 24 checkpoints, we get an NC1 curve for each seed. We take the epoch where NC1 is smallest, called `nc1_min_epoch`. Then we compare it with the grok epoch through

$$\text{ratio} = \frac{\text{nc1_min_epoch}}{\text{grok_epoch}}.$$

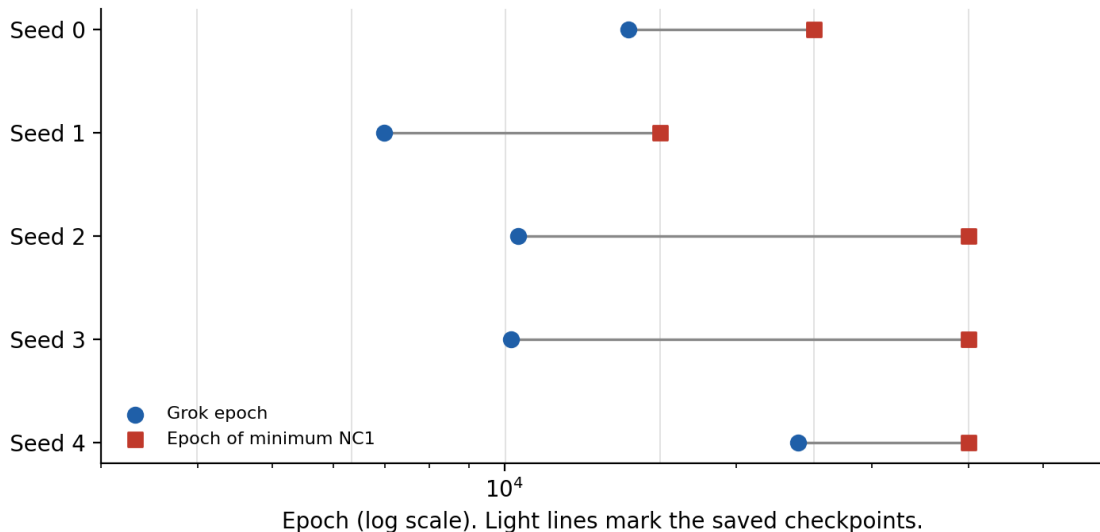
A ratio at or below 1 means that NC1 reached its minimum at or before grokking. That would make AGE a usable predictor. A ratio above 1 means NC1 arrives late.

The same two criteria are applied to every predictor (L2 Norm, Dropout, Spectral and AGE):

1. Does the signal extremum lead grok in **all 5 seeds**?
2. Is the direction of failure consistent across seeds?

7 Results on the 5 seeds

Seed	Grok epoch	nc1_min_epoch	Ratio	NC1: start → end
0	14474	25232	1.74	45.37 → 0.069
1	6988	15917	2.28	44.64 → 0.050
2	10418	39999	3.84	44.59 → 0.064
3	10193	39999	3.92	47.86 → 0.063
4	24021	39999	1.67	46.10 → 0.044



Blue circles: grok epoch. Red squares: epoch of minimum NC1. In every seed, the red square is to the right of the blue circle.

Let us read the table carefully.

- **The collapse is real.** NC1 falls from about 45 to about 0.05 in every seed. This is a reduction of roughly 660 times.
- **But it is late.** The minimum comes after grokking in 5 out of 5 seeds. The ratios lie between 1.67 and 3.92.
- **The failure is consistent.** The direction is always “late”, never “early”. This is a clear result, not noise.

So AGE fails Criterion 1, and the failure direction is the wrong one for a predictor.

8 Interpretation, and an honest caveat

8.1 What the result means

The explanation on record is this. Neural Collapse, as Papayan et al. describe it, is a *terminal-phase* phenomenon. In our model, it appears to be a *consequence* of the circuit being consolidated, not a warning sign of it. The features become tight because the model has already learned the algorithm. So NC1 tells us about the state after grokking, not before.

8.2 The caveat about the argmin

In 3 of the 5 seeds (seeds 2, 3 and 4), the minimum sits at the last checkpoint, epoch 39999. This means NC1 is still falling when training stops. Therefore `nc1_min_epoch` mostly marks “the end of training” in these seeds. The feature norm supports this view. It grows from about 1.4–1.6 at the start to tens or even hundreds at the end.

Please note the consequence. When the signal keeps decreasing until the end, the argmin *cannot* lead the grok epoch by construction. The negative verdict is therefore true for this reading of the signal. It does not say that NC1 carries no information.

8.3 A reading we have not tried

We could use the **collapse onset** instead of the argmin. For example, take the first checkpoint where $NC1 < 1$. This asks a different question: when does the fall *begin*, not when does it end? This variant is **not part of the current protocol**. It remains an open possibility. We have not tried it, and we should not report it as tried.

9 Where AGE stands in the thesis

The predictors are evaluated in a fixed order: L2 Norm, Dropout, Spectral, **AGE**, HTSR Alpha, Correlation Traps, Weight-PCA, Higher-MI, Commutator Defect.

Predictors 1 to 4 (L2 Norm, Dropout, Spectral, AGE) are now closed with negative results. The professor confirmed on 17 September 2026 that these four form the baseline. HTSR Alpha is next, and it has not been started.

What changed and what did not. The AGE code is *not deleted*. `age.py` and its plugin remain in the repository. The predictor is closed as a benchmark result, and our work has simply moved to the next predictor.

10 Summary

1. AGE computes $NC1 = \text{Tr}_W / \text{Tr}_B$ on the last-layer features at the “=” position, for all 3830 training examples, at 24 checkpoints per seed.
2. It then compares the epoch of minimum NC1 with the grok epoch.
3. NC1 collapses strongly (about $45 \rightarrow 0.05$), so the measurement itself works well.
4. The minimum comes after grokking in 5 of 5 seeds (ratios 1.67 to 3.92). AGE fails the first criterion, and in a consistent direction.
5. In three seeds the minimum is at the final checkpoint, so the argmin mostly reflects the end of training.
6. The collapse-onset reading is untried and stays open.

The main point: AGE is a correct measurement, but on this task it is a *late indicator*, not a predictor.